Programarea Calculatoarelor
Curs 1

Vlad Posea/Traian Rebedea
vlad.posea@cs.pub.ro/traian.rebedea@cs.pub.ro

About Me

- Master @Universite Lumiere Lyon, PhD @UPB
- Cercetare
  - semantic web
  - e-learning / competence modeling
  - open data
- Proiecte
  - Stagii pe Bune
  - Junio
  - &others



Obiective si continut curs

- Ciclul de dezvoltare a programului;
- Compilarea, legarea şi execuţia programelor C;
- Elemente fundamentale ale limbajului C: identificatori, cuvinte cheie, literali, comentarii, constante;
- Structura programelor C;
- Tipuri de date. Operatori. Expresii
- Instrucţiuni: expresie, compusă, void, atribuirea, decizia (if), selecţia (switch), cicluri (while, do while, for), continue, salturi disciplinate (break, return) şi nedisciplinate (goto).
- Funcţii: apel, definire, transfer prin valoare, vizibilitate
- Tablouri: declarare, tablouri uni si multidimensionale, sortare, cautare
- **Pointeri: declarare si inițializare, adresare și dereferențiere, aritmetica pointerilor, funcții care returnează pointeri**



Obiective și conținut curs

- Şiruri de caractere
- Alocarea dinamică a memoriei: Gestiunea memoriei libere (heap), malloc(), calloc(), realloc(), free(). Pointeri la pointeri. Tablouri de pointeri. Alocare dinamică pentru tablouri multidimensionale.
- Structuri: Declarare şi initializare; Accesul la membrii; pointeri la structuri. Tablouri de structuri. Atribuirea structurilor. Structuri şi funcţii. Uniuni, Câmpuri de biţi.
- Funcţii: transfer prin referinţă, Pointeri la funcţii.
- Fişiere: I/O, Fişiere text şi fişiere binare. Funcţii specifice lucrului cu fişiere
- Parametrii liniei de comandă
- Realizarea de programe complexe: Crearea de librarii statice; Vizibilitatea variabilelor.
- Funcţii cu număr variabil de parametri



Bibliografie

- ***The C Programming Language –*** Brian Kernighan and **Dennis Ritchie** (http://zanasi.chem.unisa.it/download/C.pdf)



Linkuri utile

- http://cs.curs.pub.ro – site-ul de cursuri al facultății (forum, alte resurse, teme)
- http://ocw.cs.pub.ro/courses/programare - laboratoare + regulament + alte info utile



Notare

- Laborator – 2p
- Teme de casă – 4p
- Examen final – 4p
- Condiții de trecere
  - Laborator+teme de casă>=3p &
  - Examen >=2p

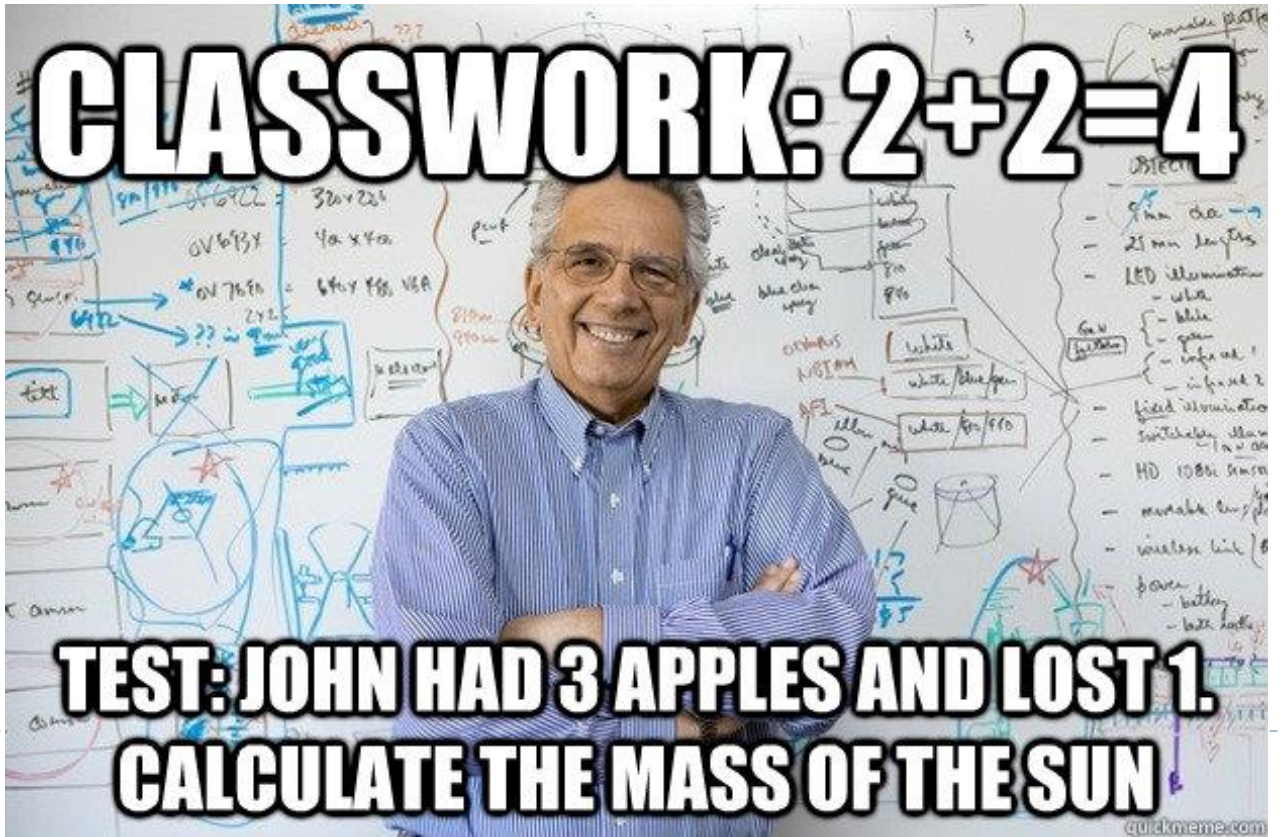


Feedback 2013 - 2016

- Probleme semnalate
  - Explicații insuficiente/nesatisfăcătoare, se avansează prea repede
    - => seminar suplimentar începând din săptămâna 2/3 pentru cei care rămân in urmă
    - frecventa ff redusa (nu stim daca il facem si in 2017)
  - Laboratoare cu prea multe activități
    - Se va pune accent la laborator mai mult pe înțelegerea materiei și mai puțin pe vânat puncte
    - La fiecare laborator se va explica la tablă o problemă și se vor rezolva alte două
  - "Vlad scrie prea urat"
    - Vlad va scrie mai mult la calculator si mai putin la tabla ☺
  - nu se aude in sala
    - vorbiti si voi un pic mai incet☺



- Probleme cu temele (prea grele, incorect/incomplet formulate, neclară evaluarea)
  - => 3 teme ce vor fi formulate/rezolvate independent și vor fi publicate din timp cu timp de rezolvare mai lung
  - temele au fost mult mai bine primite anul trecut. Vom avea mai multe subpuncte mai usoare astfel incat rezolvarea sa fie progresiva



De ce C?

Sep 2017	Sep 2016	Change	Programming Language	Ratings	Change
1	1		Java	12.687%	-5.55%
2	2		C	7.382%	-3.57%
3	3		C++	5.565%	-1.09%
4	4		C#	4.779%	-0.71%
5	5		Python	2.983%	-1.32%
6	7	↑	PHP	2.210%	-0.64%
7	6	↓	JavaScript	2.017%	-0.91%
8	9	↑	Visual Basic .NET	1.982%	-0.36%
9	10	↑	Perl	1.952%	-0.38%
10	12	↑	Ruby	1.933%	-0.03%
11	18	⇈	R	1.816%	+0.13%
12	11	↓	Delphi/Object Pascal	1.782%	-0.39%
13	13		Swift	1.765%	-0.17%
14	17	↑	Visual Basic	1.751%	-0.01%
15	8	⇊	Assembly language	1.639%	-0.78%
16	15	↓	MATLAB	1.630%	-0.20%
17	19	↑	Go	1.567%	-0.06%
18	14	⇊	Objective-C	1.509%	-0.34%
19	20	↑	PL/SQL	1.484%	+0.04%

De ce C?

Programming Language	2017	2012	2007	2002	1997	1992	1987
Java	1	2	1	1	15	-	-
C	2	1	2	2	1	1	1
C++	3	3	3	3	2	2	5
C#	4	4	7	13	-	-	-
Python	5	7	6	11	27	-	-
Visual Basic .NET	6	17	-	-	-	-	-
PHP	7	6	4	5	-	-	-
JavaScript	8	9	8	7	23	-	-
Perl	9	8	5	4	4	10	-
Assembly language	10	-	-	-	-	-	-
COBOL	25	28	17	9	3	9	9
Lisp	31	12	15	12	9	4	2
Prolog	32	31	26	16	20	11	3
Pascal	114	15	21	97	8	3	4

- http://www.tiobe.com

Caracteristici C

- Dezvoltat intre 1969-1973
- Programare imperativă
  - Programul = secvență de instrucțiuni executată de calculator
- Utilizat pentru a dezvolta sisteme de operare, aplicații embedded, aplicații telecom
- Influențează/stă la baza majorității limbajelor ”moderne”: Java, C#, Javascript, Objective-C, C++



Versiuni de C

- K&R C (1978)
- **ANSI C – 1990 (cunoscut ca C89 sau C90 sau Standard C**
- C94/C95
- C99 (1999) (tipuri de date noi, vectori de lungime variabila, …)
- C11 (2011) (suport pentru Unicode, programare paralela,…)



Etapele rezolvării unei probleme

- **Analiza** și prelucrarea datelor de intrare
  - A good programmer is someone who always looks both ways before crossing a one-way street.
  - Identificăm eventualele cazuri speciale ale datelor de intrare și le tratăm separat
    - Ex1: se cere să se citească numere ->ne asigurăm că s-au citit numere și nu alte caractere
    - Ex2: se cere să se calculeze raportul a 2 numere -> ne asigurăm că numitorul este diferit de 0
- Prelucrări necesare (algoritm de rezolvare)
- Prezentarea rezultatelor



Pseudocod

- Descriere informală a unui algoritm/program
- Exemplu:
  - Program care afișează maximul a 2 numere
    - Se citesc 2 numere intregi a,b
    - Dacă a este mai mare sau egal cu b
      - Afișează a
    - Altfel
      - Afișează b
- După ce elaborăm pseudocodul îl scriem în limbajul de programare preferat ☺ (C)



Prezentarea rezultatelor

- Rezultatele trebuie să fie inteligibile pentru utilizator

```
vlad@vlad-virtual-machine:~$ ./a.out
4 5
5vlad@vlad-virtual-machine:~$
```

Așa nu

```
vlad@vlad-virtual-machine:~$ ./a.out
introduceti 2 numere intregi:
4 5
maximul lor este: 5
```

Așa da

- Datele cerute de la utilizator trebuie să fie clare
- Eventualele mesaje de eroare trebuie să fie clare



Compilarea și execuția programelor C

- Editare fisier sursă
- Compilare fișier sursă
  - Generarea unui fișier obiect
    - gcc –c nume_sursa.c => nume_sursa.o

```
vlad@vlad-virtual-machine:~/c1$ ls
max.c
vlad@vlad-virtual-machine:~/c1$ gcc -c max.c
vlad@vlad-virtual-machine:~/c1$ ls
max.c  max.o
```

  - gcc nume_sursa.c –o nume_executabil –alteOptiuni => nume_executabil
  - Link-editare – se generează fișierul executabil pe baza unuia sau mai multe fișiere obiect



Structura unui program C

```
/* comentarii */
//When I wrote this, only God and I understood what I was doing
//Now, God only knows
/* includere fisiere antet (biblioteci) */
#include <stdio.h>

/* definitii si declarații globale*/
int maxim(int a, int b);

int main()
{
/* definiții și declarații locale */
/* secvență instrucțiuni */
return 0; /* finalizarea execuției + întoarcere rezultat*/
}
```



Hello world

```
/* includem biblioteca standard I/O */
#include <stdio.h>

/* functia main – intoarce un rezultat intreg */
int main()
{
        printf("Hello world!\n");
        return 0;
}
```

```
{acelasi program in pascal}
Program HelloWorld(output);
begin
        writeln('Hello, world!');
end.
```



Sintaxa C – elemente de bază

- Limbaj case-sensitive
  - Printf(”hello world”) nu este același lucru cu printf(”hello world”)
- Cuvintele cheie se scriu cu litere mici
- Identificatorii pot conține litere, cifre, underscore (_), nu pot incepe cu cifră
- Comentarii:
  - // comentariu pe o singura linie
  - /* comentariu care se poate întinde pe mai multe linii*/



Variabile

- Def: un mod de a referi o locație de memorie utilizată într-un limbaj de programare http://cplus.about.com/od/introductiontoprogramming/g/variabledefn.htm
- Pentru a lucra cu mai multe date trebuie să le reținem în memoria calculatorului
- Variabilele sunt folosite pentru a regăsi locul unde am scris datele în memorie
- Variabile simple: au o singură valoare la un moment dat



Variabile (2)

- Caracterizate de
  - Nume – identificatorul variabilei
  - Tip – tipul variabilei
  - Valoare
  - Adresa
- Ex: int x=5;
- int= tipul variabilei
- x=numele variabilei
- 5=valoarea variabilei
- &x = adresa din memorie unde se află stocată valoarea variabilei cu numele x.



Tipuri de date simple	Explanation	Descriptor
char	Cel mai mic tip adresabil. Contine caracterele de baza. Are valorice cu sau fara semn. In general are 1 (un) octet/ 8 biti (mai corect are CHAR_BIT biti)	%c
signed char	De aceeasi dimensiune cu char dar garantat cu semn	%c
unsigned char	De aceeasi dimensiune cu char dar garantat fără semn => are valori intre $[0,\ 2^{CHAR_BIT}-1]$	%c (or %hhu for numerical output)
short short int signed short signed short int	Tip de date intreg care are valori minim intre [−32767,+32767] deci cel putin 16 biti	%hi
unsigned short unsigned short int	La fel ca short dar fara semn	%hu
int signed signed int	Principalul tip intreg. Are valori minim intre [−32767,+32767] dar in general este reprezentat pe 32 de biti	%i sau %d
unsigned unsigned int	La fel cu int dar fara semn	%u
long long int signed long signed long int	*Intreg lung*. Are valori minim intre [−2147483647,+2147483647] deci cel putin 32 de biti	%li
unsigned long unsigned long int	La fel ca long dar fara semn	%lu

long long long long int signed long long signed long long int	*Are minim 64 de biti. Specificat in C99*	%lli
unsigned long long unsigned long long int	La fel ca long long dar fara semn	%llu
float	Tip de date real, dimensiune nespecificata	%f
double	Tip de date real precizie dubla, dimensiune nespecificata	%lf



Tipuri de date simple (2)

- C vs. Pascal

C	Pascal
short int long	integer
float double	real
char	char
int /* C has no boolean type */	boolean

- nr bytes pentru fiecare tip de date – operatorul sizeof

```
int main()
{
printf("dimensiune char: %d, dimensiune int: %d, dimensiune short: %d,\n"
"dimensiune float: %d, dimensiune double: %d\n", sizeof(char), sizeof(int), sizeof(short),
 sizeof(float), sizeof(double));
return 0;

}
```

```
vlad@vlad-virtual-machine:~/c1$ gcc size.c -o size
vlad@vlad-virtual-machine:~/c1$ ./size
dimensiune char: 1, dimensiune int: 4, dimensiune short: 2,
dimensiune float: 4, dimensiune double: 8
```

Input/Output (I/O)

- /*discutate oarecum mai devreme intrucat sunt necesare la laborator*/
- int printf (const char * format, ...);
- parametri:
  - sir de caractere ce poate conține descriptori de format
  - expresii
- descriptor de format:
  - %[flags][width][.precision][length]specifier
  - http://www.cplusplus.com/reference/cstdio/printf/



I/O (2)

specifier	Output	Example
d *or* i	Signed decimal integer	392
u	Unsigned decimal integer	7235
o	Unsigned octal	610
x	Unsigned hexadecimal integer	7fa
X	Unsigned hexadecimal integer (uppercase)	7FA
f	Decimal floating point, lowercase	392.65
e	Scientific notation (mantissa/exponent), lowercase	3.9265e+2
E	Scientific notation (mantissa/exponent), uppercase	3.9265E+2
g	Use the shortest representation: %e or %f	392.65
G	Use the shortest representation: %E or %F	392.65
c	Character	a
s	String of characters	sample
p	Pointer address	b8000000
n	Nothing printed. The corresponding argument must be a pointer to a signed int. The number of characters written so far is stored in the pointed location.	
%	A % followed by another % character will write a single % to the stream.	%



exemple printf

```
printf("valoarea lui x este: %-4.2f\n",3.14);
printf("x=%i, y=%f, x=%o, x=%#x\n",15,3.14,15,15);
printf("c= %c, c=%d\n",'%','%');
printf("sir de caractere: %s\n", "ana are mere");
printf("\\ \" \' \n");
```

```
valoarea lui x este: 3.14
x=15, y=3.140000, x=17, x=0xf
c= %, c=37
sir de caractere: ana are mere
\ " '
```

- secvente escape (caractere speciale)

\a	alert (bell) character	\\	backslash
\b	backspace	\?	question mark
\f	formfeed	\'	single quote
\n	newline	\"	double quote
\r	carriage return	*ooo*	octal number
\t	horizontal tab	\x*hh*	hexadecimal number
\v	vertical tab		

scanf

- int scanf (const char * format, ...);
- http://www.cplusplus.com/reference/cstdio/scanf/
- similar cu printf
- se transmite ca parametru adresa variabilei citite
- intoarce ca rezultat numărul de valori citite cu succes
- citește într-un buffer – stdin.



```c
int main()
{
        int a, b;
        char c;
        for(;;)
        {
                printf("Introduceti 2 numere intregi\n");
                if(scanf("%d%d",&a,&b)==2)
                        break;
                else
                        while(c=getchar()!='\n' && c!=EOF);


        }
        printf("am citit 2 numere: %d si %d\n",a,b);
        return 0;
}
```

```
vlad@vlad-virtual-machine:~/c1$ ./a.out
Introduceti 2 numere intregi
3 a
Introduceti 2 numere intregi
4 2 3
am citit 2 numere: 4 si 2
```

```
if(scanf("%d%d",&a,&b)==2)
        break;
```

- scanf este o functie. rezultatul sau este numarul de valori citite corect conform descriptorilor.
- daca am reusit sa citim corect 2 valori intrerupem iteratia cu break



```
while(c=getchar()!='\n' && c!=EOF);
```

- se citeste un caracter si se salveaza in variabila c.
- rezultatul atribuirii c=getchar() este chiar caracterul citit si acesta se compara cu \n
- se iese cand s-a intalnit \n sau EOF (character special care semnifica faptul ca nu se mai poate citi din sursa)

